Solution: Find the Duplicate Number

Problem Statement

We are given an unsorted array containing 'n+1' numbers taken from the range 1 to 'n'. The array has only one duplicate but it can be repeated multiple times. **Find that duplicate number without using any extra space.** You are, however, allowed to modify the input array.

Example 1:

```
Input: [1, 4, 4, 3, 2]
Output: 4
```

Example 2:

```
Input: [2, 1, 3, 3, 5, 4]
Output: 3
```

Example 3:

```
Input: [2, 4, 1, 4, 4]
Output: 4
```

Solution

This problem follows the **Cyclic Sort** pattern and shares similarities with Find the Missing Number. Following a similar approach, we will try to place each number on its correct index. Since there is only one duplicate, if while swapping the number with its index both the numbers being swapped are same, we have found our duplicate!

Code

Here is what our algorithm will look like:

Java

```
class Solution {

  public int findNumber(int[] nums) {
    int i = 0;
    while (i < nums.length) {
      if (nums[i] != i + 1) {  // Check if the current element is in its correct positic
        if (nums[i] != nums[nums[i] - 1])  // Check if there is a duplicate at the curre
          swap(nums, i, nums[i] - 1);  // Swap elements to their correct positions
        else // We have found the duplicate
          return nums[i]; // Return the duplicate number
      } else {
        i++; // Move to the next element
      }
    }

    return -1; // No duplicate found
  }

  private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
```

Run Code Result Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(n).

Space Complexity

The algorithm runs in constant space O(1) but modifies the input array.

Similar Problems

Problem 1: Can we solve the above problem in O(1) space and without modifying the input array?

Solution: While doing the cyclic sort, we realized that the array will have a cycle due to the duplicate number and that the start of the cycle will always point to the duplicate number. This means that we can use the fast & the slow pointer method to find the duplicate number or the start of the cycle similar to Start of LinkedList Cycle.

Java

```
class Solution {

  public int findDuplicate(int[] arr) {
    int slow = 0, fast = 0;
    do {
      slow = arr[slow];
      fast = arr[arr[fast]];
    } while (slow != fast);

    // find cycle length
    int current = arr[slow];
    int cycleLength = 0;
    do {
      current = arr[current];
      cycleLength++;
    } while (current != arr[slow]);

    return findStart(arr, cycleLength);
  }

  private static int findStart(int[] arr, int cycleLength) {
```

Run Code Result Stdout

You must run your code first

Execute

The time complexity of the above algorithm is O(n) and the space complexity is O(1).

← Previous
Find The Duplicate Number (Easy)

Next →
Find All Duplicate Numbers (Easy)

☐ Mark as Completed